

CSSCM 2026 COHORT

Part K - Full Instructor Solution

Multicomponent Chemical Analysis with NumPy

Director	Elliot S. Menkah (PhD)
Tutors	Emmanuel Oppong (MPhil); Obed Adu-Gyamfi (BSc)

Problem

A laboratory mixture contains three absorbing species A, B and C. Measurements at three wavelengths produce the following simultaneous equations:

$$2C_A + C_B + 3C_C = 9.5$$

$$C_A + 3C_B + 2C_C = 8.5$$

$$3C_A + 2C_B + C_C = 10.0$$

The task is to use NumPy to determine the three unknown concentrations and then verify that the calculated concentrations reproduce the experimental values.

1. Convert the Equations to Matrix Form

A linear system can be written compactly as $\mathbf{EC} = \mathbf{A}$, where $\mathbf{E}$ contains the coefficients, $\mathbf{C}$ contains the unknown concentrations, and $\mathbf{A}$ contains the measured values.

$$\mathbf{E} = \begin{bmatrix} 2 & 1 & 3 \\ 1 & 3 & 2 \\ 3 & 2 & 1 \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} C_A \\ C_B \\ C_C \end{bmatrix}, \quad \mathbf{A} = \begin{bmatrix} 9.5 \\ 8.5 \\ 10.0 \end{bmatrix}$$

$$\mathbf{E} \mathbf{C} = \mathbf{A}$$

2. Import NumPy

```
# Import the NumPy library.  
# 'np' is the conventional short name used for NumPy.  
import numpy as np
```

NumPy provides the array and linear-algebra tools needed for this calculation.

3. Create the Coefficient Matrix

```
# Create the coefficient matrix E.  
# Each ROW corresponds to one equation.  
# Each COLUMN corresponds to one unknown:  
# column 0 -> C_A, column 1 -> C_B, column 2 -> C_C.  
E = np.array([  
    [2.0, 1.0, 3.0],  
    [1.0, 3.0, 2.0],  
    [3.0, 2.0, 1.0]  
)  
  
# Display the matrix so that we can inspect it.  
print("Coefficient matrix E:")  
print(E)  
  
# E should contain 3 rows and 3 columns.  
print("Shape of E:", E.shape)
```

The expected shape is (3, 3). This is a two-dimensional NumPy array.

4. Create the Experimental Vector

```
# Store the values on the right-hand side of the equations.  
# These are the experimental values we want the model to reproduce.
```

```
A = np.array([9.5, 8.5, 10.0])

print("Experimental vector A:")
print(A)

# A is a one-dimensional vector containing three values.
print("Shape of A:", A.shape)
```

The expected shape is (3,), meaning a one-dimensional array with three elements.

5. Solve $EC = A$

```
# Solve the simultaneous linear equations.
# np.linalg.solve(E, A) finds the vector C for which:
#
#           E @ C = A
#
# The returned array contains [C_A, C_B, C_C].
C = np.linalg.solve(E, A)

print("Concentration vector C:")
print(C)
```

The calculated concentration vector is approximately:

$C = [2.05556, 1.22222, 1.38889]$

Therefore:

Species	Calculated concentration
A	2.05556
B	1.22222
C	1.38889

6. Print Each Concentration Clearly

```
# C[0] is the first element: concentration of species A.
# C[1] is the second element: concentration of species B.
# C[2] is the third element: concentration of species C.

print("Species A concentration =", round(C[0], 5))
print("Species B concentration =", round(C[1], 5))
print("Species C concentration =", round(C[2], 5))
```

Using `round(..., 5)` changes only how many decimal places are displayed; it does not change the original values stored in C.

7. Verify the Solution

Solving the equations is not the final step. A good numerical calculation should be checked. Since the original equation is $EC = A$, multiply the coefficient matrix by the calculated concentration vector.

```
# Matrix multiplication is performed with the @ operator.
# If the solution is correct, calculated_A should match A.
calculated_A = E @ C

print("Calculated values:")
print(calculated_A)

print("Experimental values:")
print(A)
```

Expected result:

```
Calculated values:
[ 9.5  8.5 10. ]

Experimental values:
[ 9.5  8.5 10. ]
```

Because the calculated vector reproduces the experimental vector, the numerical solution is verified.

8. What the Verification Means Mathematically

For the first equation, NumPy evaluates approximately:

$$2(2.05556) + 1(1.22222) + 3(1.38889) = 9.5$$

For the second equation:

$$1(2.05556) + 3(1.22222) + 2(1.38889) = 8.5$$

For the third equation:

$$3(2.05556) + 2(1.22222) + 1(1.38889) = 10.0$$

The small decimal representations shown on screen are rounded versions of the underlying floating-point values. NumPy retains more precision internally.

9. Complete Commented Python Solution

The complete Part K program can be saved and run as a normal Python script:

```
import numpy as np

# -----
# PART K: MULTICOMPONENT CHEMICAL ANALYSIS
# -----

# E is the coefficient matrix.
# Rows represent the three equations.
# Columns represent the coefficients of C_A, C_B and C_C.
E = np.array([
    [2.0, 1.0, 3.0],
    [1.0, 3.0, 2.0],
    [3.0, 2.0, 1.0]
])

# A stores the measured values on the right-hand side.
A = np.array([9.5, 8.5, 10.0])

# Show the numerical system that will be solved.
print("Coefficient matrix E:")
print(E)

print("\nExperimental vector A:")
print(A)

# Solve E @ C = A.
# The result C contains the three unknown concentrations.
C = np.linalg.solve(E, A)

print("\nCalculated concentration vector:")
print(C)

# Access the individual elements of the solution vector.
print("\nIndividual concentrations:")
print("Species A =", round(C[0], 5))
print("Species B =", round(C[1], 5))
print("Species C =", round(C[2], 5))

# Verify the answer by substituting the solution back into
# the matrix equation. This performs matrix multiplication.
calculated_A = E @ C

print("\nVerification:")
print("Calculated values  :", calculated_A)
print("Experimental values:", A)

# np.allclose checks whether two numerical arrays are equal
# within a small floating-point tolerance.
print("\nSolution verified:", np.allclose(calculated_A, A))
```

10. Expected Terminal Output

```
Coefficient matrix E:
[[2. 1. 3.]
 [1. 3. 2.]
 [3. 2. 1.]]

Experimental vector A:
[ 9.5  8.5 10. ]

Calculated concentration vector:
[2.05555556 1.22222222 1.38888889]

Individual concentrations:
Species A = 2.05556
Species B = 1.22222
Species C = 1.38889

Verification:
Calculated values  : [ 9.5  8.5 10. ]
Experimental values: [ 9.5  8.5 10. ]
```

Solution verified: True

Key Learning Points

1. A system of chemical equations can be represented using arrays and matrices.
2. `np.array()` creates the numerical matrix and vectors.
3. `np.linalg.solve(E, A)` solves the linear system directly.
4. `E @ C` performs matrix multiplication.
5. Always verify a numerical solution by substituting it back into the original mathematical model.
6. `np.allclose()` is useful for checking floating-point numerical results.